Karlsruhe University of Applied Sciences

Department of Computer Science and Business Information Systems

Business Information Systems

BACHELOR´S THESIS

Efficiency vs. Performance in Green AI: An Empirical Investigation of Energy Consumption, Carbon Footprint, and Predictive Accuracy of Classification Algorithms at Varying Dataset Sizes.

Author	Eron Qorri
Matriculation number	85383
Workplace	Hochschule Karlsruhe
First Advisor	Prof. Dr. rer. nat. Christine Preisach
Second Advisor	Prof. Dr. Reimar Hofmann
Closing Date	31 July, 2026

31 July, 2026

Chairman of the Examination Board

Contents

Chapter 1

Introduction

1.1 Problem statement and societal relevance

The current trend in machine learning is moving towards increasingly large models and datasets [1]. Although deep learning approaches often achieve the highest predictive accuracy, their energy demand and associated CO_2 emissions increase disproportionately [2, 3]. In an era marked by strict sustainability guidelines and rising energy costs, a critical question arises: At what point is a computationally "simple" model ecologically more viable than a highly complex one?

The primary objective of this thesis is to determine whether and when it is worthwhile to deploy more complex models, considering not only their predictive accuracy but also their associated energy consumption. To achieve this, a Logistic Regression, a Random Forest, an XGBoost model and a Multilayer Perceptron are compared under equal conditions across three datasets of varying sizes. This comparative analysis aims to provide a clearer understanding of the thresholds at which simpler models represent the better choice. Additionally, the study investigates how modifying a Multilayer Perceptron architecture, specifically varying the number of neurons and hidden layers, directly impacts energy consumption and CO_2 emissions.

To address the problem statement and achieve the outlined objectives, this thesis investigates the overarching question: *How does the relationship between energy consumption and predictive accuracy change for Logistic Regression, Random Forest, XGBoost, and Multilayer Perceptron when model complexity and dataset size are systematically varied?*

Specifically, the following sub-questions are examined:

- How do energy consumption (in kWh) and CO_2 emissions change when scaling a deep learning architecture (variation of neuron count)?
- Under which conditions (dataset size, model complexity) does the resource consumption of complex models exceed the achievable accuracy gain compared to simpler algorithms?

- How does the energy consumption differ between CPU-based training (Random Forest) and GPU-accelerated training (XGBoost, MLP) for the same classification task?

1.2 Motivation: When does a simpler model pay off?

1.3 Research questions and objectives

1.4 Structure of the Thesis

The remainder of this thesis is structured as follows: Chapter 2 provides the theoretical background, distinguishing between Green AI and Red AI, and introducing the relevant sustainability and classification metrics. Chapter 3 details the methodology, including dataset selection and the experimental hardware setup. Chapter 4 outlines the implementation process, focusing on data preprocessing, model training, and the logging of resource consumption. The empirical results are analyzed in Chapter 5, evaluating both performance and resource utilization across scaling dataset sizes. Chapter 6 discusses the findings, highlighting the economic and environmental implications. Finally, Chapter 7 concludes the thesis and provides actionable recommendations for sustainable machine learning practices.

$$S = \frac{F_1}{1 + \lambda_1 \cdot t_{\text{scaled}} + \lambda_2 \cdot c_{\text{scaled}}}$$

Chapter 2

Theoretical Background

This chapter establishes the theoretical foundations required to understand the empirical study presented in the remainder of this thesis. It first frames the research within the broader debate between *Red AI* and *Green AI*, then formalizes the concepts of energy consumption and carbon emissions in machine learning. Subsequently, the relevant classification algorithms are described from a purely theoretical standpoint, followed by the metrics used to evaluate their predictive performance, the validation strategy, hyperparameter optimization techniques, and finally the hardware concepts that are central to comparing CPU- and GPU-based training. All implementation details are deliberately deferred to Section 5.3.

Diese Einleitung ist nur ein Platzhalter — am Ende des Chapters nochmal anpassen, sobald die finalen Sections feststehen.

2.1 Green AI and Red AI

Übergreifende Section-Einleitung: kurz motivieren, warum dieses Framing für die Arbeit zentral ist (es liefert das ökologische Vokabular für RQ und Metriken). Verweis auf Section 2.5 (EWF1) und Section 2.2.

2.1.1 The Rise of Red AI

Definition Red AI nach Schwartz et al.: Forschung, die Genauigkeit primär durch immer mehr Compute, Daten und Parameter erkauft. Skalierungsgesetz von Compute pro SOTA-Paper (Verdopplung alle 3.4 Monate, [4] bzw. analog). Kosten-Trends: Strubell et al. (Transformer-Training), GPT-Skalierung. Wichtig: nicht moralisieren, sondern Phänomen sachlich beschreiben.

2.1.2 Green AI as a Counter-Movement

Definition Green AI: Forschung, die Effizienz neben Genauigkeit als Erstklass-Metrik behandelt. Kernforderung von [1]: Reporting von FLOPs / Energie / CO_2 als Standardpraxis. Abgrenzung Green AI vs. "AI for Green" (KI *für* Nachhaltigkeit, z.B. Smart Grids) — letzteres ist NICHT Thema dieser Arbeit. Erwähnung verwandter Initiativen: ML CO_2 Impact, MLCO2 Calculator, Green Software Foundation.

2.1.3 Efficiency as a First-Class Metric

Konzeptueller Übergang: Wie operationalisiert man "Effizienz"? Diskussion möglicher Proxies — FLOPs (hardware-unabhängig, aber abstrakt), Parameteranzahl (irreführend bei Sparse Models), Trainingszeit (hardware-abhängig), Energie in kWh, CO_2eq. Trade-offs jeder Metrik kurz benennen. Begründung, warum diese Arbeit auf kWh + CO_2eq + Trainingszeit setzt — leitet direkt zu Section 2.5 über. KEIN Vorgriff auf EWF1-Formel (das gehört zu Metrics).

2.2 Energy and Carbon Emissions in Machine Learning

Section-Einleitung: Brücke von Konzept (Effizienz) zu Messung. Erklären, dass Energieverbrauch und Emissionen *nicht dasselbe* sind — Energie ist eine physikalische Größe (kWh), Emissionen hängen vom Strommix ab.

2.2.1 Energy Consumption of Computing Hardware

Physikalische Grundlagen: Leistung (Watt) vs. Energie (Joule, kWh). Beziehung $E = P \cdot t$. Welche Komponenten verbrauchen Strom beim ML-Training: CPU, GPU, RAM, ggf. Speicher / Netzwerk. Konzept TDP (Thermal Design Power) — was es ist und was es *nicht* ist (kein durchschnittlicher Verbrauch, sondern thermische Auslegungsgrenze). Idle vs. Last-Verbrauch. Wichtig für später: GPUs haben deutlich höhere Peak-Leistung, aber auch deutlich höheren Durchsatz — Effizienz pro Sample ist entscheidend.

2.2.2 From Energy to CO_2 Equivalents

Definition Carbon Intensity (gCO_2eq/kWh), Abhängigkeit vom Strommix (Kohle vs. Wind/Solar/Atom). Konzept CO_2-Äquivalent: warum nicht nur CO_2, sondern auch CH_4, N_2O etc. über GWP (Global Warming Potential) zusammengefasst werden [3]. Formel: Emissions $= E \cdot \text{CarbonIntensity}$. Beispielhafter Vergleich: 1 kWh in Frankreich ($\tilde{5}0$ g) vs. Polen ($\tilde{7}00$ g). Verweis auf Section 5.1.1 für den konkreten deutschen Strommix.

2.2.3 Measurement Approaches

Drei Hauptansätze für Energiemessung gegenüberstellen: (1) **Hardware-Counter**: RAPL für Intel/AMD CPUs (Linux), NVML für NVIDIA-GPUs. Genauigkeit, aber OS-/Hardware-abhängig [5]. (2) **TDP-basierte Schätzung**: Annahme eines festen Prozentsatzes (z.B. 50 %) der TDP. Ungenau, aber überall verfügbar. (3) **Externe Wattmeter**: Hardware-Steckdosenmessung. Genau, aber nur Gesamtsystem, keine Komponentenauflösung. Klarstellen, warum diese Arbeit auf Software-Ebene misst (Reproduzierbarkeit, keine Hardware-Investition, granular nach Komponenten). Limitationen ehrlich benennen.

2.2.4 CodeCarbon

Diese Subsection existiert bereits inhaltlich — bei der Überarbeitung des Chapters den bestehenden Text leicht straffen und sicherstellen, dass er logisch auf Section 2.2.3 aufbaut (CodeCarbon ist eine konkrete Realisierung des Software-Messansatzes). Label `sec:theoretical_codecarbon` unbedingt beibehalten — wird aus Section 5.5 referenziert.

Hier kurz erklären: CPU Package Power Sensor vs. TDP-basierte Schätzung. TDP (Thermal Design Power) ist die maximale Wärmeleistung unter Dauerlast — kein Echtzeitwert. Der CPU Package Power Sensor liefert dagegen den tatsächlichen Momentanverbrauch direkt vom Hardware-Monitor-Chip (ausgelesen via LibreHardwareMonitor). Relevant weil CodeCarbon auf Windows auf TDP-Schätzung zurückfällt — der empirische Unterschied wird in Section 5.5.3 gezeigt.

CodeCarbon is an open-source Python library for measuring the carbon footprint of code running on local hardware. Emissions are calculated as the product of two factors: the energy consumed by the hardware in kWh, and the carbon intensity of the local electricity grid in gCO_2eq/kWh. Carbon intensity is determined based on the country in which the experiment is run, using data from Our World in Data. The resulting emissions are expressed in kilograms of CO_2-equivalents ($kgCO_2eq$), a standardized measure that accounts for the global warming potential of different greenhouse gases [3].

Power consumption is sampled at 15-second intervals. Rather than relying on instantaneous power readings, *CodeCarbon* derives average power from accumulated hardware energy counters. Instead, it computes the energy delta between two consecutive measurements and divides by the elapsed time to derive average power. This approach is more robust against short-term fluctuations and provides a stable average over each measurement interval [6]. For NVIDIA GPUs, energy is read directly from hardware counters via the `nvidia-ml-py` library, providing accurate device-level measurements. RAM is estimated by *CodeCarbon*, specifically using a flat value of 5 watts per DIMM slot.

CPU measurement depends on the operating system and processor. On Linux, CodeCarbon reads energy directly from *RAPL* (Running Average Power Limit) hardware counters, which support both Intel and AMD processors [5]. On Windows, however, *RAPL* is not accessible. *CodeCarbon* therefore falls back to a TDP-based estimation: it identifies the CPU model from a database of over 2000 processors and assumes an average consumption of 50% of TDP. On systems where direct CPU energy measurement is unavailable, this TDP-based fallback may introduce inaccuracies, as TDP represents peak power rather than typical operating consumption.

2.3 Foundations of Supervised Classification

Section-Einleitung: kurze Einordnung, warum Klassifikation als Task gewählt wurde (siehe Section 4.2) — hier nur die theoretischen Grundbegriffe.

2.3.1 Supervised Learning

Formale Definition: gegeben Trainingsmenge $\{(x_i, y_i)\}_{i=1}^{n}$, lerne Funktion $f : \mathcal{X} \to \mathcal{Y}$. Abgrenzung zu Unsupervised und Reinforcement Learning in 1–2 Sätzen. Begriff *Generalisierung*, Overfitting/Underfitting in einem Satz (Detail kommt in Validation Strategies).

2.3.2 Binary and Multiclass Classification

Definition binär ($|\mathcal{Y}| = 2$) vs. multiklass ($|\mathcal{Y}| > 2$). Bezug zu den drei Datasets (Wine = 3-Klassen, Credit/HIGGS = binär). One-vs-Rest und Softmax-Erweiterung als Standardstrategien für lineare/Softmax-Modelle. Hier noch keine Modell-Details — die kommen in Section 2.4.

2.3.3 Loss Functions for Classification

Cross-Entropy-Loss formal einführen — sowohl binär (Log-Loss) als auch multiklass (Categorical Cross-Entropy / mlogloss). Formeln angeben. Kurz erwähnen, warum Cross-Entropy gegenüber MSE bei Klassifikation bevorzugt wird (Gradientenverlauf, probabilistische Interpretation). Querverweise: wird in MLP (Section 2.4.5) und XGBoost (Section 2.4.4) als Lernziel verwendet.

2.4 Classification Algorithms

Section-Einleitung: Modelle in Reihenfolge wachsender Komplexität (Linear → Tree → Ensemble → Boosting → NN), wie auch in Section 4.3 motiviert. WICHTIG: hier rein theoretische Beschreibung — keine Konfigurationsdetails, keine Hyperparameter-Werte, keine Library-Erwähnungen. Implementation in Section 5.3.

2.4.1 Logistic Regression

Lineares Modell mit Sigmoid-Aktivierung: $P(y = 1|x) = \sigma(w^\top x + b)$. Sigmoid-Funktion und ihre Eigenschaften. Optimierung über Maximum Likelihood / Cross-Entropy-Minimierung. Solver kurz erwähnen (LBFGS, SAG, Newton-CG) — aber NUR theoretisch, der konkrete `sag`-Solver wird in Section 5.3.1 begründet. Erweiterung auf Multinomial Logistic Regression (Softmax). Vor- und Nachteile: interpretable, schnell, aber linear → keine Feature-Interaktionen.

2.4.2 Decision Trees

Grundlage für RF und XGBoost — daher zuerst eingeführt. CART-Algorithmus: rekursive binäre Splits, Greedy-Auswahl nach Gini-Impurity oder Entropie. Formeln für Gini und Entropie. Konzept Tree Depth, Leaf Nodes, Pruning. Stärken (nicht-linear, interpretable) und Schwächen (Overfitting bei tiefen Bäumen, instabil gegenüber kleinen Datenänderungen) — letzteres motiviert direkt Ensembles.

2.4.3 Random Forest

Bagging-Prinzip nach [7]: n Bootstrap-Samples → n Bäume → Majority Vote / Mittelwert. Zusätzliche Randomisierung über zufällige Feature-Subsets pro Split (mtry / max_features). Warum das die Varianz reduziert (Bias-Varianz-Zerlegung kurz). Begriff Out-of-Bag (OOB) Error. Parallelisierbarkeit über Bäume hinweg → Ankerpunkt für CPU-Multi-Threading-Diskussion in Section 2.8. Skalierungsverhalten: superlinear in n bei tiefen Bäumen — Begründung, warum RF auf HIGGS in Section 5.3.2 ausgeschlossen wurde, schon hier theoretisch andeuten.

2.4.4 Gradient Boosting and XGBoost

Boosting-Idee: sequentiell schwache Lerner addieren, jeder fittet Residuen / Gradienten des bisherigen Ensembles. Formal: Funktionsraumgradientenabstieg [8]. Übergang zu XGBoost [9]: 2nd-Order-Gradienten (Hessian), Regularisierungsterm im Loss ($\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$), Shrinkage (learning rate), Column Subsampling. Histogram-basierte Split-Findung (gpu_hist) als Brücke zu GPU-Beschleunigung — wichtig für RQ "CPU vs. GPU".

2.4.5 Multilayer Perceptron

Section-Einleitung: kurz historisch (Perceptron [10], dann Mehrschicht via Backprop [11]). Danach modular in Subsubsections — der derzeit in Section 5.3.4 stehende theoretische Block ("BatchNorm normalisiert Aktivierungen über den Mini-Batch ...") soll hierhin verschoben werden, dort nur noch die Implementation übrig lassen.

Perceptron and Feedforward Architecture

Einzelner Perceptron als gewichtete Summe + Aktivierung. Stack zu Schichten: $h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)})$. Universal Approximation Theorem in 1–2 Sätzen erwähnen. Architekturparameter: Anzahl Hidden Layers, Neuronen pro Layer — direkter Bezug zu RQ "Skalierung der Architektur".

Activation Functions

Sigmoid, Tanh, ReLU vergleichen. Vanishing-Gradient-Problem von Sigmoid/Tanh, daher Dominanz von ReLU [12]. Eigenschaften ReLU: $\max(0, x)$, sparse Aktivierungen, billiger Gradient. Kurze Erwähnung von Varianten (Leaky ReLU, GELU) — aber begründen, warum in dieser Arbeit Standard-ReLU genutzt wird.

Backpropagation and Gradient Descent

Chain Rule für Gradienten durch das Netz (Backpropagation). Stochastic Gradient Descent (SGD) → Mini-Batch SGD → Adam [13] mit adaptiven Momenten. Formel Adam-Update kompakt. Konzept Learning Rate als wichtigster Hyperparameter (motiviert spätere Tuning-Suchräume).

Batch Normalization

Inhalt aus Section 5.3.4 hierher verschieben: Normalisierung der Aktivierungen pro Mini-Batch zu Mittelwert 0 / Varianz 1, dann affine Transformation mit lernbaren γ, β. Effekt: Reduktion des Internal Covariate Shift, schnellere Konvergenz, leichte Regularisierung [14].

Dropout

Inhalt aus Section 5.3.4 hierher verschieben: zufälliges Deaktivieren von Neuronen mit Wahrscheinlichkeit p während des Trainings, volle Aktivierung beim Inference (skaliert). Interpretation als implizites Ensemble [15]. Warum BN + Dropout zusammen funktionieren (mit Vorsicht — bekannte Wechselwirkungen kurz erwähnen).

Early Stopping as Regularization

Konzept: Training abbrechen, sobald Validation-Loss nicht mehr sinkt. Theoretische Begründung als impliziter Regularisierer (begrenzt effektive Modellkapazität). Patience-Parameter erklären. Querverweis: konkrete Implementation in Section 5.3.4.

2.5 Evaluation Metrics

Section-Einleitung: drei Metrik-Familien — (i) Klassifikationsgüte, (ii) Ressourcenverbrauch, (iii) zusammengesetzte Effizienzmetriken. Wird aus Section 4.4 referenziert.

2.5.1 Confusion Matrix and Accuracy

Confusion Matrix für binär (TP, TN, FP, FN) und Erweiterung auf multiklass. Accuracy = $(TP + TN)/N$ bzw. Anteil korrekter Vorhersagen. Limitation bei Class Imbalance demonstrieren mit kleinem Beispiel (z.B. 99 % Mehrheitsklasse → 99 % Accuracy mit trivialem Klassifikator) — motiviert F1.

2.5.2 Precision, Recall and F1-Score

Definitionen: Precision = $TP/(TP + FP)$, Recall = $TP/(TP + FN)$, F1 = harmonisches Mittel. Warum harmonisches statt arithmetisches Mittel (penalisiert Extremwerte). Macro / Micro / Weighted F1 unterscheiden — Weighted F1 = pro Klasse gewichtet mit Support, nutzt diese Arbeit. Begründung für Weighted F1 bei den hier verwendeten Datasets (Credit ist imbalanced, HIGGS leicht imbalanced).

2.5.3 Resource Metrics

Trainingszeit (Sekunden), Energie (kWh), CO_2eq (kg), Inference-Latenz (Sekunden / Sample). Querverweis auf Section 2.2 für Definitionen. Hier nur knapp die Rolle als Evaluationsmetrik begründen.

2.5.4 Composite Efficiency Metrics

Generelle Idee zusammengesetzter Metriken: ein Score, der Performance gegen Kosten abwägt. Beispiele aus der Literatur: Energy-Efficiency Ratio, Performance-per-Watt, FLOPs-per-Sample. Hier theoretischer Hintergrund — die konkrete EWF1-Definition für diese Arbeit steht in Section 4.4, hier nur die methodische Grundlage und Schwächen solcher Composite-Metriken (Gewichtungswillkür, Hardware-Abhängigkeit) diskutieren.

2.6 Validation Strategies

Section-Einleitung: warum nicht einfach auf Trainingsdaten evaluieren (Overfitting-Risiko, Generalisierung). Wird aus Section 4.5 referenziert.

2.6.1 Hold-Out Train-Test Split

Einfachste Strategie: X in train/test aufteilen (z.B. 80/20). Vorteile (schnell, einfach), Nachteile (hohe Varianz bei kleinen Datasets, Performance-Schätzung hängt stark vom konkreten Split ab).

2.6.2 K-Fold Cross-Validation

K-Fold-Algorithmus: Daten in k Folds, k-mal trainieren, jeweils ein Fold als Test. Mittelwert über Folds als Performance-Schätzer, Standardabweichung als Stabilitätsmaß. Trade-off k: kleines $k \rightarrow$ schneller, höherer Bias; großes k (LOOCV) $\rightarrow$ niedriger Bias, sehr teuer. $k = 5$ und $k = 10$ als Standardwerte [16]. Rechtfertigung für $k = 5$ in dieser Arbeit verlinken auf Section 4.5.

2.6.3 Stratified Sampling

Stratifizierte Splits erhalten Klassenverhältnisse in jedem Fold. Wichtig bei imbalanced Datasets (Credit, HIGGS). Kurz erklären, dass scikit-learn's StratifiedKFold dies automatisch tut — hier nur das Prinzip, keine API-Details.

2.7 Hyperparameter Optimization

Section-Einleitung: aus Section 4.6 wird hierher referenziert für die genaue theoretische Erklärung von Bayesian Optimization / Optuna.

2.7.1 Hyperparameters vs. Parameters

Klare Abgrenzung: Parameter = während Training gelernt (Gewichte), Hyperparameter = vor Training festgelegt (Lernrate, Tiefe, Anzahl Bäume). Suchraum als Cartesian Product der HP-Wertebereiche. Begriff Validation-Set / inneres CV für HP-Selektion.

2.7.2 Grid Search and Random Search

Grid Search: erschöpfende Suche über vorgegebenes Gitter. Skaliert exponentiell mit Anzahl HP — konkretes Rechenbeispiel passt zu Section 4.6 (5 HPs $\times$ 10 Werte $\times$ 4 Modelle $\times$ 3 Datasets = 150,000 Evaluationen). Random Search nach [17]: zufällige Stichprobe aus Suchraum, in höherdimensionalen Räumen oft effizienter als Grid. Beide Verfahren *informationslos* (lernen nichts aus vergangenen Trials) — das motiviert Bayes.

2.7.3 Bayesian Optimization

Sequentielles modellbasiertes Optimieren: Surrogatmodell (z.B. Gaussian Process oder TPE) approximiert die Objective, Acquisition Function (Expected Improvement, UCB) wählt nächsten Punkt. Exploration vs. Exploitation. TPE (Tree-structured Parzen Estimator) nach [18] kurz beschreiben — modelliert $p(x|y)$ statt $p(y|x)$, gut für mixed/conditional search spaces. Komplexitätsvorteil ggü. Grid Search bei vielen HPs.

2.7.4 Optuna as a Practical Framework

Optuna [19] als define-by-run-Framework: Suchraum wird im Objective-Code dynamisch konstruiert (statt vorab als Config). Standardsampler ist TPE. Pruner-Konzept (frühes Abbrechen schlechter Trials, z.B. MedianPruner) — nur theoretisch, ob diese Arbeit Pruning nutzt steht in Section 5.4. Reproducibility über Sampler-Seed. KEINE Code-Snippets hier — die gehören in Chapter 5.

2.8 Hardware Considerations for Machine Learning

Section-Einleitung: motiviert die CPU-vs-GPU-Forschungsfrage aus theoretischer Sicht. Konkrete verwendete Hardware steht in Section 5.1.1, hier nur die Konzepte.

2.8.1 CPU Architecture for ML Workloads

CPU-Eigenschaften: wenige starke Cores, hohe Single-Thread-Leistung, große Caches, latenzoptimiert. Multi-Threading via OpenMP / Intel MKL für Matrixoperationen. Warum CPUs für RF-Bagging gut geeignet sind (Bäume sind unabhängig parallelisierbar, aber jeder Baum selbst ist sequentiell mit irregulären Speicherzugriffen).

2.8.2 GPU Computing and CUDA

GPU-Architektur: tausende einfache Cores, SIMT-Execution, durchsatzoptimiert. CUDA als NVIDIA-Programmiermodell. Warum Matrixmultiplikation (und damit NN-Training) ideal für GPUs ist: dichte parallele Operationen mit regelmäßigem Speichermuster. Memory-Hierarchie GPU (HBM/GDDR vs. Shared Memory) kurz erwähnen, nur soweit für Verständnis nötig.

2.8.3 Algorithmic Suitability for CPU vs. GPU

Zusammenfassend: welche Algorithmen profitieren wovon? Logistic Regression: meist CPU (Datentransfer-Overhead lohnt sich nicht). Random Forest: CPU (irreguläre Tree-Traversal). XGBoost: beides möglich (`hist` auf CPU, `gpu_hist` auf GPU) — dieser Vergleich ist Kern einer RQ. MLP: GPU klar überlegen ab nicht-trivialer Größe. Energiebetrachtung anschließen: GPU hat höhere Peak-Leistung, aber bei passenden Workloads deutlich mehr Sample/J → bessere Energieeffizienz, NICHT zwangsläufig weniger Energieverbrauch absolut. Diese Theorie wird in Chapter 6 empirisch geprüft.

2.9 Chapter Summary

Kurzes Fazit (max. ein Absatz): die wichtigsten theoretischen Bausteine für die folgenden Chapters sind benannt — Green-AI-Framing, Energie-/Emissions-Definitionen, Algorithmen-Theorie, Metriken, Validation, HPO und Hardware. Brücke zu Chapter 3 (Related Work) und Chapter 4 (Experimental Design).

Chapter 3

Related Work

Chapter 4

Experimental Design

This chapter outlines the experimental design used to evaluate the ecological efficiency of different machine learning classification algorithms. It describes the datasets, models, evaluation metrics, hyperparameter tuning and measurement setup that form the basis of this empirical analysis.

4.1 Research Questions

The main research question that this study aims to answer is: *How does the trade-off between energy consumption and predictive performance vary across Random Forest, XGBoost and MLP models when the complexity of the models and the size of the dataset are systematically varied?* The following sub-questions will also be addressed in the course of this thesis:

- Under what conditions do the resource consumption of more complex models exceed the achievable accuracy gain over simpler baselines?
- How does energy consumption and CO2 output scale with increasing neural network complexity (number of layers and neurons)?
- How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?
- How does reducing the number of training instances on large datasets affect model accuracy and energy consumption across different algorithms?
- How accurate are software-based energy estimation tools compared to direct hardware sensor measurements, and how does this affect reported CO_2 emissions?

Letzte Unterfrage in der Diskussion ausführlicher behandeln: CodeCarbon unterschätzt CPU-Verbrauch empirisch um Faktor 2–9× (gemessen via LibreHardwareMonitor CPU Package Power Sensor). Begründung: TDP-basierte Schätzung skaliert linear mit CPU-Auslastung, bildet aber weder Boost-Verhalten noch Uncore-Komponenten ab. Konkrete Messwerte als Beleg vorhanden (z.B. LogReg Credit: 6.6 W CodeCarbon vs. 57.8 W HardwareMonitor; LogReg Higgs: Faktor ∼2.3×).

RQ4 (Scaling): Kurz das experimentelle Design für den Subset-Run beschreiben. Die Kernidee: statt verschiedener Datasets wird *derselbe* Datensatz (HIGGS) in fünf Größen aufgeteilt (100k, 500k, 1M, 5M, 11M Instanzen via `TEST_NROWS` Umgebungsvariable). Alle Modelle außer Random Forest werden mit denselben Hyperparametern (aus dem HIGGS-Tuning) auf jeder Größe trainiert. So wird der Dataset-Größen-Effekt isoliert, ohne dass unterschiedliche Hyperparameter oder Feature-Räume den Vergleich verfälschen. Ergebnis: CO2, Trainingszeit und F1 als Funktion von nrows für alle Modelle.

4.1.1 Break-Even Analysis

To determine at which dataset size GPU training becomes more energy-efficient than CPU training for XGBoost, a break-even analysis was designed as part of this study. The analysis is based on nine HIGGS subsets ranging from 1,000 to 11,000,000 instances, using identical hyperparameters for both devices. This controlled setup isolates the effect of dataset size without confounding factors from different feature spaces or hyperparameter configurations.

The relationship between dataset size and CO_2 emissions is modelled as a power law, fitted separately for CPU and GPU by applying a linear regression to $\log_{10}(\text{nrows})$ versus $\log_{10}(CO_2)$. A log-log fit is used because emissions scale super-linearly with data size, and a linear model would systematically misrepresent the relationship. The break-even point n_{BE} is then derived analytically as the intersection of the two fitted lines:

$$\log_{10}(n_{BE}) = \frac{b_{\text{GPU}} - b_{\text{CPU}}}{a_{\text{CPU}} - a_{\text{GPU}}} \tag{4.1}$$

where a and b are the slope and intercept of each fit. Below n_{BE}, CPU training is more energy-efficient; above it, GPU training is. The results are presented in Chapter 6.

To answer these questions multiple classification algorithms and datasets were used in the experimental phase of this empirical analysis.

4.2 Datasets

Dataset size is a primary driver of both computational demand and energy consumption during model training [20]. To evaluate how this relationship varies across different scales, three classification datasets were selected that span several orders of magnitude in size. Classification was chosen as the task type since it is well-established across all selected models and allows for direct comparison of predictive performance via standardized metrics such as accuracy and F1-score. The datasets are summarized in Table 4.1.

Table 4.1: Overview of datasets used in this analysis

Dataset	**Instances**	**Features**	**Classes**	**Scale**
Wine [21]	178	13	3	Small
Default of Credit Card Clients [22]	30,000	23	2	Medium
HIGGS [23]	11,000,000	28	2	Large

The *Wine* dataset contains 178 instances across 13 features, where each feature describes a chemical property of wines from the same region in Italy, derived from three different cultivars.

The *Default of Credit Card Clients* (hereafter: *Credit*) dataset contains 30,000 instances across 23 features and focuses on predicting payment default among credit card clients in Taiwan.

The *HIGGS* dataset is the largest dataset used in this study, comprising 11,000,000 instances across 28 features, seven of which are high-level features derived by physicists to help discriminate between the two classes [24]. The classification goal is to distinguish between signal events produced by Higgs bosons and background noise processes.

These three datasets vary significantly in size, enabling a systematic evaluation of how model performance and energy consumption scale across different levels of data complexity.

4.3 Models

Selecting models of varying complexity is central to answering the research questions of this study. A model that achieves competitive accuracy with significantly lower energy consumption represents a more ecologically efficient choice, but this trade-off can only be quantified if the comparison spans a meaningful range of complexity. For this reason, five classification models were selected, ranging from a linear baseline to tree-based ensemble methods and a neural network, covering low, medium, and high computational demand respectively.

Logistic Regression serves as the lightweight baseline in this study. Despite its simplicity as a linear classifier, it often achieves competitive accuracy, making it well

Table 4.2: Overview of classification models used in this study

Model	Type	Device	Role
Logistic Regression [25]	Linear	CPU	Baseline
Random Forest [7]	Ensemble	CPU	Mid-complexity
XGBoost [9]	Gradient Boosting	CPU & GPU	Mid-complexity
Multilayer Perceptron	Neural Network	GPU	High-complexity

suited to quantifying whether the additional resource consumption of more complex models yields a meaningful gain in predictive performance.

Random Forest was selected as a mid-complexity representative of tree-based ensemble methods, as extensively described in Section 2.x . Relevant to this study, Random Forest does not support GPU-accelerated training and is therefore run exclusively on the CPU.

Building on the tree-based approach, XGBoost was chosen as a second mid-complexity model, with the key distinction that it also supports GPU-accelerated training. This makes it possible to directly compare the energy consumption and training time of CPU- and GPU-based tree ensemble methods under identical conditions, which is central to one of the research questions of this study.

The MLP was selected as the most complex model, representing the deep learning category. Its highly configurable architecture, particularly the number of hidden layers and neurons per layer, makes it well suited for the architectural complexity experiments conducted in this study, where these parameters are systematically varied to assess their impact on predictive performance and energy consumption. A more detailed description of the MLP architecture is provided in Section 2.x.

Together, the selected models cover a broad spectrum of algorithmic complexity, from a linear baseline to gradient boosting and deep learning, enabling a meaningful comparison of both predictive performance and energy efficiency across fundamentally different model types. The following section defines the metrics used to evaluate these dimensions.

4.4 Evaluation Metrics

To answer the research questions of this study, two categories of metrics were employed: classification performance and resource consumption. Accuracy and weighted F1-Score were selected to measure predictive performance, as they provide a standardized basis for comparing models across datasets of varying class distributions, as further described in Section 2.5. Energy consumption in kWh and CO_2 emissions in kg were chosen as the primary resource metrics, directly reflecting the ecological cost of model training. It should be noted that all experiments were conducted on a single hardware setup, which is described in detail in Section 5.1.1.

Training time in seconds was included as a secondary resource metric, as it captures computational demand independently of hardware-specific power draw. Furthermore, inference time was measured across all experimental setups to account for the operational footprint. In large-scale deployment scenarios, the cumulative energy demand of repeated predictions can often rival or even exceed the initial training costs [26]. Together, these metrics enable a direct assessment of whether gains in predictive performance justify the associated increase in resource consumption.

Inspired by the efficiency framing proposed in Schwartz et al., a custom composite metric was developed for this study, named the *Efficiency-Weighted F1* (EWF1), defined as:

$$\text{EWF1} = \frac{F_1}{1 + \lambda_1 \cdot \hat{t} + \lambda_2 \cdot \hat{c}} \tag{4.2}$$

where $\hat{t}$ and $\hat{c}$ denote the min-max normalized training time and CO2 emissions per dataset respectively. The weights were set to $\lambda_1 = 0.5$ and $\lambda_2 = 1.0$, assigning greater importance to carbon emissions than training time, reflecting the ecological focus of this study. Higher scores indicate a more favorable trade-off between predictive performance and resource consumption.

It should be noted that both $\hat{t}$ and $\hat{c}$ are inherently hardware- and location-dependent, as training time varies with the underlying hardware configuration and CO2 emissions are tied to the carbon intensity of the local electricity grid [1]. Consequently, EWF1 scores reflect the ecological efficiency of the specific experimental setup used in this study and may not generalize directly to other hardware or energy infrastructures.

4.5 Validation Strategy

To obtain reliable and generalizable performance estimates, a cross-validation strategy was employed rather than a single train-test split, as described in Section 2.x.

Specifically, K-Fold cross-validation was applied with $k = 5$ folds, a value widely recommended in the literature as a practical balance between computational cost and estimation stability [16]. Each model is trained and evaluated five times on non-overlapping subsets of the data, and the final performance score is averaged across all folds. This approach is consistent across all models and datasets, ensuring comparability of results.

To ensure reproducibility, a fixed random state was used for all cross-validation splits, model initialization, and data shuffling. The specific value and its technical implementation are described in Section 5.1.2.

4.6 Hyperparameter Tuning

To ensure a fair comparison across all models, hyperparameter tuning was applied to each model prior to evaluation. Without tuning, default parameters tend to favor certain model families over others, potentially skewing the comparison in terms of both predictive performance and resource consumption [27].

Hyperparameter optimization was performed using Optuna [19], an optimization framework based on Bayesian search (see Section 2.x).

Hier in 02 die genaue Erklärung

Grid search was considered but deemed infeasible given the scale of experiments: assuming five hyperparameters per model with ten candidate values each, evaluated across three datasets and four models, a full grid search would require approximately 150,000 evaluations. Optuna was therefore configured to run 50 trials per model per dataset, striking a balance between tuning thoroughness and resource consumption, the latter being a relevant factor given that tuning runs contribute to the overall energy budget of this study, as described in Section 4.4.

Weighted F1-score was used as the optimization objective, as it accounts for class imbalance more robustly than accuracy, providing a more reliable estimate of model quality across datasets with varying class distributions.

Tuning was performed separately for each dataset, as the datasets differ substantially in size, feature space, and class distribution. A globally tuned model would not represent optimal performance on any individual dataset and would therefore introduce bias into the comparison.

Chapter 5

Implementation and Measurement

This chapter describes how the experimental pipeline was built and how measurements were collected. It covers the hardware and software environment, the data loading architecture, each model implementation, the hyperparameter tuning setup, and the emissions measurement infrastructure including the custom CPU power monitoring solution.

5.1 Experimental Setup

5.1.1 Hardware Setup

A critical decision in the experimental design was the selection of the computing platform on which all experiments would be conducted. Two options were considered: the university's shared computing infrastructure or a local machine. The latter was ultimately chosen, as it provides several key advantages for this study. First, it ensures complete reproducibility, since all experiments are executed on identical hardware configuration, eliminating interfering variables that could arise from shared resource conflicts or system variability. Second, it provides consistent availability and direct control over system configuration, which is particularly important given the extended runtime of the large-scale experiments (particularly on the 11M-instance HIGGS dataset). Third, local execution facilitates fine-grained performance monitoring and measurement of energy consumption, a primary objective of this study.

The hardware platform selected for all experiments is a consumer-grade desktop system with the following specifications: an NVIDIA RTX 4070 graphics processing unit (12 GB GDDR6 memory, 5,888 CUDA cores), an AMD Ryzen 7 5700X processor (8 cores / 16 threads, base clock 3.6 GHz), and 32 GB of DDR4 RAM. The system runs Windows 11 with all code implemented in Python 3.13.0.

The choice of hardware reflects a deliberate trade-off between experimental scope and practical feasibility. The RTX 4070 represents a mid-to-high-end consumer GPU with sufficient compute capability to execute GPU-accelerated training for XGBoost and MLP

Table 5.1: Hardware specification of the experimental system

Component	Specification
CPU	AMD Ryzen 7 5700X (8 cores / 16 threads, 3.6 GHz base)
GPU	NVIDIA RTX 4070 (12 GB GDDR6, 5,888 CUDA cores)
RAM	32 GB DDR4
Operating System	Windows 11

models in reasonable time, while remaining accessible to many practitioners. The Ryzen 7 5700X provides adequate CPU performance for baseline models (Logistic Regression, Random Forest) and CPU-based XGBoost training. Together, this configuration enables direct comparison of CPU- and GPU-accelerated training on the same platform, which is central to answering one of the research questions (*How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*).

It is important to note that energy consumption and CO2 emissions measured in this study are specific to this hardware configuration and the local electricity grid, and may not directly generalize to other systems or geographic regions. However, the relative performance comparisons (e.g., the energy efficiency ranking of different algorithms) are more likely to generalize, as they reflect algorithmic properties rather than hardware specifics alone.

5.1.2 Software Environment

All experiments were implemented in Python 3.13.0. The main libraries used are described below.

Scikit-learn [28] served as the foundation of the evaluation pipeline. It provided the `LogisticRegression` and `RandomForestClassifier` implementations, as well as `KFold` and `cross_validate` for cross-validation, `StandardScaler` for feature normalization, `make_pipeline` for chaining preprocessing and model steps, and `f1_score` and `make_scorer` for evaluation metrics.

The XGBoost library [9] was used for the gradient boosting model, as it provides a highly optimized implementation that supports both CPU and GPU training.

For the MLP, PyTorch [29] was used as the deep learning backend. To integrate the PyTorch model into the scikit-learn evaluation pipeline, the Skorch library [30] was used, which wraps PyTorch models in a scikit-learn-compatible interface.

Hyperparameter tuning for all models except Logistic Regression was performed using Optuna, as described in Section 4.6.

Carbon emissions and energy consumption were tracked using CodeCarbon [31], which is described in detail in Section 5.5.

To ensure reproducibility, a fixed random seed of 42 was used for all data splits, cross-validation folds, and model initialization. A `requirements.txt` file is provided

so that the exact library versions used in this study can be reproduced.

The implementation code was developed with the assistance of Claude Code [32], an AI-powered development tool by Anthropic. All generated code was reviewed, tested, and integrated by the author.

5.1.3 Reproducibility and Evaluation Design

To ensure fully reproducible results across all experiments, a global constant `RANDOM_STATE` with a value of 42 is used in every model script. It is passed to the `KFold` splitter and all model constructors. For the MLP, the seed is additionally set via `torch.manual_seed(RANDOM_STATE)` to guarantee deterministic weight initialization.

The evaluation relies on `KFold` with five splits and shuffling enabled, executed through `cross_validate()`. Performance is measured using a scoring dictionary that tracks both accuracy and weighted F1 score via `make_scorer(f1_score, average="weighted")`. Final scores are the mean across all five folds.

5.2 Dataset Integration

The three datasets used in this study, *Wine*, *Credit*, and *HIGGS*, were introduced in Section 4.2. This section describes how they are integrated into the software architecture, covering the central configuration file, the data loading pipeline, and dataset-specific constraints.

5.2.1 Configuration Architecture

To ensure a consistent foundation across all model scripts, a central configuration file (`config.py`) was created. It serves as a single source of truth for all dataset-related settings, so that each script accesses the same parameters without redundant definitions.

For each dataset, the configuration stores the file path, the name of the target column, and an optional row limit (`nrows`). The row limit was particularly relevant for the HIGGS dataset, which contains 11 million instances, allowing experiments to be run on smaller subsets where needed. Beyond these common fields, the configuration also captures dataset-specific properties. For example, the Credit dataset includes a header row that must be skipped during loading, and the Wine dataset does not contain column names in the source file, which are therefore defined in the configuration directly. Additionally, a label offset is applied to the Wine target column, as its class labels are encoded as 1, 2, and 3 rather than starting at 0, which is required by the classifiers used in this study.

The global constants `RANDOM_STATE` and `CV_FOLDS` are also defined here, ensuring that all scripts use identical values for random seeding and cross-validation, which is essential for the comparability of results.

5.2.2 Data Loading Pipeline

To read the datasets from their respective files, a custom `load_data` method was implemented. Depending on the dataset, the appropriate loading mechanism is selected. The *Wine* and *Credit* datasets are loaded using pandas' `read_csv` function. For the *Higgs* dataset, `read_parquet` is utilized. This was a deliberate decision to save storage space by storing the large *Higgs* dataset as a Parquet file rather than a standard CSV.

Following the initial loading phase, the *Higgs* data is reduced to a representative subset using a sampling approach controlled by the `nrows` parameter. Once the data size is managed, any columns specified in the configuration file are dropped from the respective dataset. The data is then split into the feature matrix X and the target vector y. If an offset is defined in the config, which is currently only the case for the *Credit* dataset, it is applied as described in the previous section. Finally, the method returns X and y for further use in the subsequent scripts. This process is visualized in Figure 5.1.

Due to the substantial scale of the *Higgs* dataset, several constraints were implemented to ensure the stability of the execution environment. Training a `Random Forest` model on this specific data led to unmanageable processing times. For this reason, the `Random Forest` algorithm is explicitly bypassed for the *Higgs* dataset and the program terminates using `sys.exit(0)` to prevent system crashes. This allows the process to stay within the boundaries of the available hardware resources.

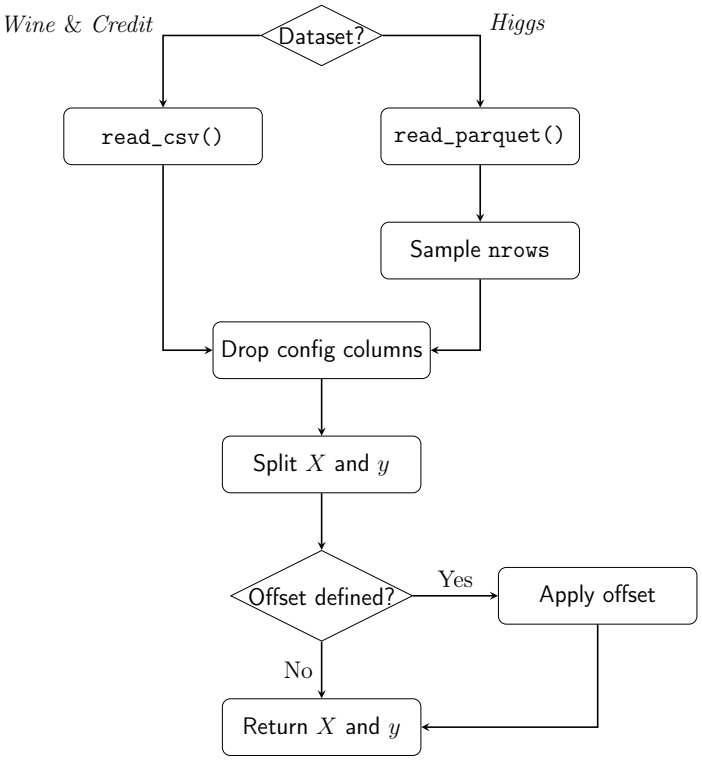

Figure 5.1: Visualization of the data loading pipeline and preprocessing steps within the `load_data()` method.

5.3 Model Implementations

As outlined in Section 4.3, the following classification algorithms were employed to address the research questions mentioned previously. This section therefore details the specific implementation of each model.

5.3.1 Logistic Regression

The first baseline model is a standard logistic regression. To ensure the model processes all features on an equal scale, the algorithm is embedded in a pipeline starting with a `StandardScaler`. This initial scaling step is crucial for the stability and overall performance of the subsequent classification.

The regression is configured to use the Stochastic Average Gradient (SAG) algorithm by setting the `solver` parameter to `sag`. This specific solver is highly optimized for large datasets. It significantly reduces the memory footprint and required training time, which is especially important when evaluating the extensive *Higgs* dataset. To ensure proper convergence, the maximum number of iterations is capped at 1000 using the `max_iter` parameter.

This model setup deliberately excludes any hyperparameter optimization. Instead, it acts as a stable baseline to evaluate the raw computational time without the extra overhead of complex search processes.

It is worth noting that the SAG solver is single-threaded and runs on a single CPU core, resulting in roughly 10% utilisation on the 8-core system used in this study. The `cross_validate()` call also uses no `n_jobs` parameter, so the five folds run sequentially. Each model in this study runs with its natural parallelisation behaviour; the implications of this for energy comparability are discussed in Section 5.5.3.

5.3.2 Random Forest

The second baseline algorithm is the Random Forest classifier. This method belongs to the family of ensemble learning techniques. Unlike logistic regression, this algorithm does not require prior feature scaling. To maximize computational efficiency, the execution is fully parallelized across all available CPU cores by setting the `n_jobs` parameter to `-1`. This configuration allows the processor to run at maximum capacity, which directly influences the subsequent energy consumption tracking and overall runtime measurements.

As established in Section 5.2.2, Random Forest is explicitly excluded from the *Higgs* dataset due to unmanageable training times at 11M instances. Including it would produce extreme runtimes without contributing meaningful insight into the scaling behaviour studied here.

To ensure maximum predictive capabilities for the remaining data, the model

configurations were optimized using Optuna. The optimization process specifically targeted maximizing the weighted F1 score. For strict reproducibility, the final parameter combinations for the *Wine* and *Credit* datasets are documented in Table 5.2.

Table 5.2: Optimized hyperparameters for the Random Forest classifier obtained via Optuna optimization.

Parameter	*Wine*	*Credit*
`n_estimators`	221	381
`max_depth`	5	9
`min_samples_split`	8	10
`min_samples_leaf`	3	2
`max_features`	`sqrt`	`None`

hier noch die korrekten werte nach dem finalen run eintragen

5.3.3 XGBoost

The next tree-based ensemble model is the XGBoost classifier. This model serves as a direct point of comparison to the Random Forest. A primary reason for including XGBoost in this study is that it allows for a direct comparison of computational efficiency between CPU and GPU processing. To achieve this, the configuration dictionaries are kept identical for both executions. The only difference is the hardware device parameter, which is set to use the GPU. This controlled setup ensures a precise evaluation of the energy and emissions generated by the exact same model architecture on different hardware. This setup directly addresses the third research question: *How does energy consumption differ between CPU-based and GPU-based training for equivalent tasks?*.

The break-even analysis described in Section 4.1.1 is computed in the plotting pipeline by reading all HIGGS subset rows from `results.csv` and fitting the log-log model separately for CPU and GPU using `numpy.polyfit`. The intersection is calculated analytically using Equation (4.1).

Break-Even-Wert aus finalem Run eintragen und in Results referenzieren.

Additionally, XGBoost has a specific technical requirement for handling target variables. While the Random Forest algorithm natively adapts to different class structures, XGBoost requires explicit instructions for its learning objective. As mentioned in Section 4.2, the *Wine* dataset consists of three distinct categories. Therefore, the objective parameter is set to `multi:softmax`, paired with the `mlogloss` evaluation metric and a specified number of classes. In contrast, the *Credit* and *Higgs* datasets contain only two categories. Consequently, their objective is configured as `binary:logistic` and evaluated using the standard `logloss` metric.

Similar to the previous baseline algorithm, the parameters tailored to each dataset were determined using the Optuna optimization framework. Furthermore, the algorithm

processes the unscaled feature matrix X directly. It also uses the globally defined random state parameter to guarantee a deterministic execution across all hardware setups. The resulting configurations used for the final training phase are detailed in Table 5.3.

Table 5.3: Optimized hyperparameters for the XGBoost classifier across all datasets.

Parameter	*Wine*	*Credit*	*Higgs*
`objective`	`multi:softmax`	`binary:logistic`	`binary:logistic`
`eval_metric`	`mlogloss`	`logloss`	`logloss`
`num_class`	3	N/A	N/A
`n_estimators`	464	108	475
`max_depth`	7	3	8
`learning_rate`	0.0139	0.2878	0.2428
`subsample`	0.7790	0.9640	0.7100
`colsample_bytree`	0.7500	0.8430	0.9368
`min_child_weight`	5	7	2

5.3.4 Multilayer Perceptron

As discussed in Section 4.3, a Multilayer Perceptron was selected to represent the neural network family. The implementation uses PyTorch as the primary deep-learning framework, combined with the Skorch library. Skorch wraps the PyTorch model in an interface that is fully compatible with the scikit-learn ecosystem. This is necessary because the entire evaluation logic, including cross-validation and pipeline construction, relies heavily on scikit-learn.

A dynamic architecture was chosen to ensure the network structure directly reflects the optimal parameters found during the Optuna tuning process. At runtime, the script reads the best parameter set from a JSON file generated during the tuning phase. It extracts the number of hidden layers via `n_layers` and the neuron count for each individual layer via `layer_i`. These values are then assembled into a list called `layer_sizes`. This list is subsequently passed to the `MLPModule` class. This class constructs the full network dynamically upon instantiation, making the architecture entirely data-driven.

The `MLPModule` inherits from `nn.Module` and builds the hidden layers by iterating over the layer size list. Each hidden block consists of `nn.Linear` followed by `nn.BatchNorm1d`, a ReLU activation, and a dropout layer (see Section 2.4.5 for the theoretical background on these components). After all hidden blocks, a single linear output layer maps to the number of target classes with no activation, since `CrossEntropyLoss` expects raw logits.

To minimize the overall training time, the implementation checks for CUDA availability using `torch.cuda.is_available()`. The algorithm runs on the GPU if one is

present and automatically falls back to the CPU otherwise. PyTorch requires highly specific numeric types, and neural networks react sensitively to implicit type mismatches. Therefore, the feature matrix X is explicitly cast to `float32` and the target vector y to `int64` before the training begins.

The neural network is embedded within a scikit-learn `Pipeline` together with a `StandardScaler`. Since Multilayer Perceptrons lack built-in scale invariance, standardizing the inputs to a mean of zero and a unit variance is necessary to keep the gradient flow stable. The `NeuralNetClassifier` provided by Skorch receives all architecture parameters through a specific naming convention. It uses the `module__*` prefix to forward keyword arguments directly to the constructor of the `MLPModule`. The Adam optimizer and the `CrossEntropyLoss` criterion are used consistently across all datasets. The learning rate and the batch size are taken directly from the tuning results and are therefore tailored to each dataset. A fixed random seed is applied using `torch.manual_seed` to guarantee exact reproducibility across all iterations.

The maximum number of training epochs is set to 200. This value acts as an upper boundary rather than a strict target. An early stopping callback (`skorch.callbacks.EarlyStopping`) monitors the validation loss after each individual epoch. It halts the training process immediately if no improvement is observed for 10 consecutive epochs. This mechanism prevents unnecessarily long training runs and provides an additional layer of protection against overfitting. Consequently, the upper limit of 200 epochs is rarely reached in practice.

5.4 Hyperparameter Optimization

Building on the design outlined in Section 4.6, this section describes how Optuna was integrated into the evaluation pipeline and which search spaces were defined for each model. The tuning logic is encapsulated in three dedicated scripts (`tune_xgb.py`, `tune_rfc.py`, and `tune_mlp.py`), one per tunable model. All three follow the same structural pattern and differ only in the model that is instantiated and the search space that is queried from the trial object. The target dataset is selected via a command-line argument, and the number of trials is read from the `N_TRIALS` environment variable, with a default of 50. This separation between configuration and code allows individual tuning runs to be triggered without modifying the source files. The search spaces used across all three tuning scripts are summarized in Table 5.4. Integer and continuous parameters define an inclusive range, while categorical parameters list the discrete choices available to the sampler.

After the dataset has been loaded through the shared `load_data()` function described in Section 5.2.2, each script constructs a `KFold` splitter using the same global constants `CV_FOLDS` and `RANDOM_STATE` that are also applied during the final evaluation. Reusing the identical splitter ensures that the F1 score returned to Optuna is computed

Table 5.4: Hyperparameter search spaces defined in the Optuna tuning scripts.

Model	Parameter	Type	Range / Values
Random Forest	`n_estimators`	integer	[100, 500]
	`max_depth`	integer	[3, 20]
	`min_samples_split`	integer	[2, 20]
	`min_samples_leaf`	integer	[1, 10]
	`max_features`	categorical	{`sqrt`, `log2`, `None`}
XGBoost	`n_estimators`	integer	[100, 500]
	`max_depth`	integer	[3, 8]
	`learning_rate`	float (log)	[0.01, 0.3]
	`subsample`	float	[0.6, 1.0]
	`colsample_bytree`	float	[0.6, 1.0]
	`min_child_weight`	integer	[1, 10]
MLP	`n_layers`	integer	[1, 4]
	`layer_i` (per layer)	categorical	{64, 128, 256, 512}
	`dropout_rate`	float	[0.0, 0.5]
	`lr`	float (log)	$[10^{-4}, 10^{-1}]$
	`batch_size`	categorical	{1024, 4096, 8192}

under the same validation conditions as the metrics reported later in Chapter 6, eliminating any discrepancy between the score the optimizer maximizes and the score the study ultimately reports.

The core of every script is an `objective(trial)` function that draws a candidate hyperparameter configuration from the trial object, instantiates the corresponding classifier, and returns the mean weighted F1 score obtained from a five-fold cross-validation via `cross_val_score` with `make_scorer(f1_score, average="weighted")`. This is the only output that Optuna is exposed to. Everything else, such as emissions tracking and timing, is handled outside the objective function.

Each study is created via `optuna.create_study(direction="maximize")` and parameterized with a `TPESampler` seeded with `RANDOM_STATE`. Seeding the sampler is what makes the trial sequence reproducible across re-runs. Without it, the suggested hyperparameter combinations would differ on every execution, and the resulting tuning emissions would no longer be comparable. The optimization itself is launched through `study.optimize(objective, n_trials=N_TRIALS)`.

To capture the energy footprint of the tuning process itself, the entire `study.optimize` call is wrapped in a CodeCarbon `EmissionsTracker` with a script-specific `project_name` (e.g. `tune_mlp_higgs`), so that tuning runs are clearly distinguishable from training runs in the `emissions/` directory. The same tracker is used across all three scripts and is configured identically to the trackers used during training, as described in Section 5.5.

Once a study completes, the best score, the corresponding hyperparameter dictionary,

the duration, the emissions reported by the tracker, and the trial count are written to a shared `best_params.json` file via the `save_best_params()` utility. The function performs an upsert keyed by model and dataset, so that re-running a single tuning configuration overwrites only its own entry and leaves the other results untouched. The model scripts introduced in Section 5.3 consume this file through the corresponding `load_best_params()` helper, which decouples tuning from training entirely: the final training pipeline never instantiates Optuna, it merely reads the previously persisted hyperparameter dictionary and passes it to the classifier constructor.

5.5 Emissions Measurement

Measuring energy consumption and CO_2 emissions is the central objective of this study, and the measurement infrastructure therefore required particularly careful design. The following subsections describe how CodeCarbon was integrated into each script, how measurement coverage was defined across model and dataset combinations, and what hardware-specific constraints and limitations apply to the collected values.

5.5.1 CodeCarbon Integration

Energy consumption and CO_2 emissions were tracked using *CodeCarbon* as described in Section 2.2.4. In each model script, an `EmissionsTracker` instance is created before the cross-validation loop begins and configured with two key parameters: `output_dir`, which points to the shared `emissions/` directory at the project root, and `project_name`, which uniquely identifies the run. The tracker is then started by calling `tracker.start()` immediately before the `cross_validate()` call, and stopped via `tracker.stop()` once all folds have completed. The return value of `tracker.stop()` is a single float representing the total CO_2-equivalent emissions in kilograms accumulated over the entire tracked interval. This value is what gets passed to `save_results()` and written to `results.csv`.

The placement of the tracker boundaries was a deliberate choice. Data loading and preprocessing happen before `tracker.start()`, so they are excluded from the measurement. Only the cross-validation itself, which includes all five training and evaluation passes, falls inside the tracked interval. This ensures that the reported emissions value reflects the cost of model training and evaluation exclusively, and that values are directly comparable across models and datasets without interference.

Figure 5.2 illustrates this structure. Prior to training, data is loaded and, where required, features are normalized using `StandardScaler`. Scaling was applied to Logistic Regression and MLP, as these models are sensitive to the magnitude of input features, while tree-based models (Random Forest, XGBoost) were trained on the raw input data.

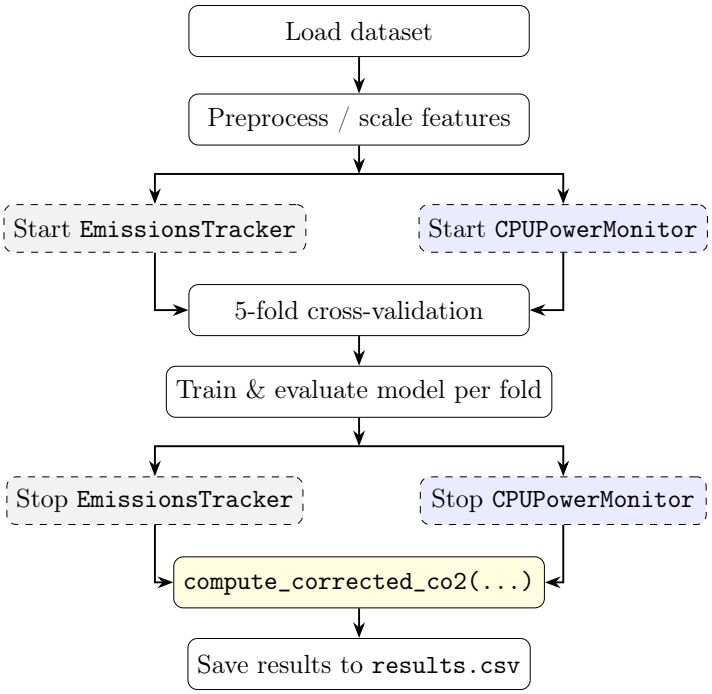

Figure 5.2: Training and emissions tracking procedure per model-dataset combination. Dashed gray boxes indicate CodeCarbon `EmissionsTracker` calls; dashed blue boxes indicate `CPUPowerMonitor` calls (background thread, 0.25 s sampling). Both run in parallel during cross-validation. After training, `compute_corrected_co2` replaces CodeCarbon's TDP-based CPU estimate with the hardware sensor value.

5.5.2 Measurement Scope and Naming

To allow fine-grained analysis of emissions across all experimental configurations, each model–dataset combination receives its own `EmissionsTracker` instance with a unique `project_name`. The naming convention follows the pattern `<model>_<dataset>`, for example `lr_wine`, `rf_credit`, or `xgb_gpu_higgs`. This scheme makes individual runs immediately identifiable in the `emissions/` directory, where CodeCarbon writes one CSV file per tracked run in addition to returning the scalar value in code. In the same manner, the tuning scripts save the results with the `<model>_<dataset>` pattern, preceded by a `tune_`, for example `tune_mlp_higgs` or `tune_xgb_cpu_wine`.

In total, the experiment produces tracked emissions records for 14 training runs (all five models across all three datasets, minus the Random Forest on HIGGS which is explicitly skipped) and up to eight tuning runs (Random Forest across two datasets, XGBoost and MLP across all three). The separate CSV files written by CodeCarbon to the `emissions/` directory serve as a raw audit trail, while the values extracted via `tracker.stop()` are the primary source for the analysis presented in Chapter 6.

5.5.3 Hardware Measurement and Limitations

The general measurement approach used by CodeCarbon, including direct GPU readout via NVML, RAM estimation per DIMM slot, and TDP-based CPU fallback on Windows, is described in Section 2.2.4. This subsection focuses on the concrete implications for the hardware used in this study.

CodeCarbon's TDP-based CPU estimation on Windows has been replaced in this study by a custom `CPUPowerMonitor` class implemented in `models/power_monitor.py`. It runs in a background thread and polls the CPU Package Power sensor every 0.25 seconds via the *LibreHardwareMonitor* library (`HardwareMonitor` Python package). This library reads the CPU Package Power sensor directly from the hardware, providing true power values rather than a TDP approximation. In practice, CodeCarbon underestimated CPU power by a factor of roughly 9× compared to the hardware sensor (e.g. 6.6 W vs. 57.8 W for Logistic Regression on the Credit dataset). The corrected CO_2 value in `results.csv` therefore replaces CodeCarbon's CPU contribution with the hardware-measured value, while GPU and RAM energy are still taken from CodeCarbon. The original CodeCarbon total is retained as `co2eq_codecarbon_kg` for reference.

This approach does, however, come with notable limitations. Because LibreHardwareMonitor requires privileged access to hardware sensors, all measurements in this study were conducted with the process running under administrator privileges. Without elevated rights, the `CPUPowerMonitor` fails to retrieve sensor data and the corrected CO_2 estimate cannot be computed.

A second limitation concerns the differences in CPU parallelisation across models. Logistic Regression, for instance, relies on the SAG solver, which executes on a single core, whereas Random Forest and XGBoost exploit multi-threading across all available threads. This disparity directly affects the instantaneous power draw: a single-threaded workload draws considerably fewer watts. However, the reduced wattage is largely offset by a proportionally longer runtime, meaning that the total energy in watt-hours may converge to a similar value. The implications of this trade-off for the comparability of emissions estimates are discussed in Chapter 7.

A secondary limitation is that CodeCarbon measures at the process level, meaning background system activity on Windows 11 contributes to the recorded energy. This effect is expected to be small relative to the dominant training workload but cannot be fully eliminated.

5.6 Inference Latency Measurement

Inference time was measured separately after training, outside the emissions tracker. For each model, a single sample from the test set was passed through the trained model and the elapsed time was recorded using `time.perf_counter()`. This provides a hardware-level estimate of per-sample prediction latency, which is relevant in deployment

scenarios where repeated inference can accumulate significant energy costs [26].

Only a single row was passed to the model rather than a full batch. This was intentional: batching would measure throughput rather than the actual time it takes to process one request, which is the more relevant figure in deployment. A single-prediction latency can be extrapolated to real-world usage, where a model may handle millions of requests and the per-sample cost accumulates into a meaningful energy expenditure.

The model used for this measurement is taken from the first cross-validation fold via `cv_results["estimator"][0]`, which is made available by passing `return_estimator=True` to `cross_validate()`. The single-row slice `X[:1]` is then passed to `trained_model.predict()`, with the elapsed time captured using `time.perf_counter()` for sub-millisecond precision.

5.7 Results Persistence and Orchestration

This section describes how experimental results are stored and how the individual model scripts are coordinated into complete experiment runs. The two concerns are closely related: the storage format determines what data is available for analysis, while the orchestration layer controls how and in what order the model scripts are executed.

5.7.1 Results Schema

The results of every model run are written to `results/results.csv`, which serves as the central data source for all subsequent analysis and visualisation. Each row represents one complete model-dataset combination and is appended to the file rather than overwriting it, so that results from multiple runs accumulate without data loss. The file is managed using Python's `csv.DictWriter`, which writes a header row on first creation and appends subsequent rows without repeating the header. A consequence of the append-only design is that re-running a configuration produces duplicate rows, which must be filtered during analysis. Table 5.5 lists all columns.

The primary emissions value is `co2eq_kg`, which combines the hardware-sensor CPU reading with CodeCarbon's GPU and RAM estimates as described in Section 5.5.3. The column `co2eq_codecarbon_kg` retains the original CodeCarbon total for reference. The difference between the two columns is examined in Chapter 7.

Inference latency is stored separately in `results/inference_time.csv`, as it is measured outside the emissions tracker and has a different structure. Like `results.csv`, it uses append-only writes via `csv.DictWriter`. Table 5.6 shows its columns.

5.7.2 Experimental Orchestration

Three orchestration scripts coordinate the full experiment. Each model script is launched as an isolated subprocess rather than being imported directly. This is important because

Table 5.5: Column schema of `results.csv`.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size; `all` for full dataset
`accuracy`	float	Mean CV accuracy
`f1`	float	Mean weighted F1 score
`co2eq_kg`	float	Corrected CO_2
`co2eq_codecarbon_kg`	float	Original CodeCarbon estimate
`cpu_power_hw_w`	float	Average CPU package power (W)
`cpu_energy_hw_wh`	float	Total CPU energy (Wh)
`training_time_s`	float	Total CV wall-clock time (s)

Table 5.6: Column schema of `inference_time.csv`.

Column	Format	Description
`timestamp`	datetime	Date and time of the run
`model`	string	Model name
`dataset`	string	Dataset name
`nrows`	int / `all`	Subset size; `all` for full dataset
`inference_time`	float	Single-row prediction latency (s)
`co2eq_kg`	float	Corrected CO_2 of the training run

CodeCarbon operates at the process level: running multiple trackers within the same process risks state leakage between measurements. Subprocess isolation ensures that each tracker starts and stops cleanly. A failed script does not abort the overall run; failures are collected and reported at the end.

`run_all.py` is the main entry point for the full experiment. It runs in two sequential phases. Phase 1 executes all tuning scripts for Random Forest, XGBoost, and MLP across their respective datasets, and saves the best parameters to `best_params.json`. Phase 2 then runs all five model scripts across all three datasets. Each script is launched via `subprocess.run()` with the working directory set to `models/`, so that relative imports resolve correctly.

The other two scripts are used for the subset-based analyses. `run_xgb_breakeven` runs XGBoost CPU and GPU directly on nine HIGGS subset sizes ranging from 1,000 to 11,000,000 rows. Unlike the other orchestration scripts, it calls the model logic directly rather than via subprocess, because it manages the emissions tracker and the cross-validation loop itself. This script provides the data for the break-even analysis in Chapter 6. `run_scaling_all_models.py` runs all models except Random Forest via subprocess across five HIGGS subset sizes (100k to 11M rows), and provides the data for the scaling analysis.

Both subset scripts control the dataset size through an environment variable `TEST_NROWS`. When set, `load_data()` reads this value and draws a stratified sam-

ple of that size instead of loading the full dataset. The value is written to the `nrows` column in `results.csv` as a concrete integer, making subset results directly filterable during analysis. The key advantage of this approach is that no changes to any model script are required; the subset size is controlled entirely from the outside.

Chapter 6

Results and Evaluation

Chapter 7

Discussion

Zur Aussagekraft des Break-Even-Points: Der log-log-Fit basiert auf 9 HIGGS-Subset-Größen (1k–11M) mit gleichen Hyperparametern — das isoliert den Dataset-Größen-Effekt sauber. Dennoch Limitierungen benennen: (1) Potenzgesetz ist eine Approximation. (2) Ergebnis gilt nur für diese Hardware und diesen Datensatz. (3) Fit empfindlich gegenüber Ausreißern an den Rändern. Break-Even-Wert als Größenordnung interpretieren, nicht als präzisen Schwellenwert.

CodeCarbon vs. HardwareMonitor comparison: discuss the systematic underestimation of CPU power by CodeCarbon on Windows (TDP-based fallback vs. sensor readout). Show the difference between `co2eq_codecarbon_kg` and `co2eq_kg` per model/dataset. Interpret what this means for reproducibility and comparability of emissions studies that rely solely on CodeCarbon.

Parallelisation trade-off: Logistic Regression (SAG solver, single core) draws fewer watts than Random Forest / XGBoost (multi-threaded), but the longer runtime may result in comparable total energy in Wh. Discuss whether watt-based or Wh-based comparison is the fairer metric for cross-model emissions comparability, and what this means for interpreting the results.

Bibliography

[1] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Commun. ACM*, vol. 63, no. 12, pp. 54–63, Nov. 2020, ISSN: 0001-0782. DOI: 10.1145/3381831 Accessed: Mar. 19, 2026.

[2] E. Strubell, A. Ganesh, and A. McCallum, *Energy and Policy Considerations for Deep Learning in NLP*, Jun. 2019. DOI: 10.48550/arXiv.1906.02243 arXiv: 1906.02243 [cs]. Accessed: Mar. 19, 2026.

[3] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, *Quantifying the Carbon Emissions of Machine Learning*, Nov. 2019. DOI: 10.48550/arXiv.1910.09700 arXiv: 1910.09700 [cs]. Accessed: Mar. 19, 2026.

[4] *AI and compute*, https://openai.com/index/ai-and-compute/, Jun. 2022. Accessed: Apr. 19, 2026.

[5] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in Action: Experiences in Using RAPL for Power Measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 1–26, Jun. 2018, ISSN: 2376-3639, 2376-3647. DOI: 10.1145/3177754 Accessed: Apr. 15, 2026.

[6] *How Power Estimation Works in CodeCarbon - CodeCarbon*, https://docs.codecarbon.io/latest/expl estimation/. Accessed: Apr. 15, 2026.

[7] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, ISSN: 1573-0565. DOI: 10.1023/A:1010933404324 Accessed: Mar. 19, 2026.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine.," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, Oct. 2001, ISSN: 0090-5364, 2168-8966. DOI: 10.1214/aos/1013203451 Accessed: Apr. 19, 2026.

[9] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '16, New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 785–794, ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785 Accessed: Mar. 19, 2026.

[10] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958, ISSN: 1939-1471. DOI: 10.1037/h0042519

[11] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Apr. 19, 2026.

[12] G. Hinton, V. Nair, and Y. Rachmad, "Rectified Linear Units Improve Restricted Boltzmann Machines Vinod Nair," vol. 27, pp. 807–814, Jun. 2010.

[13] D. P. Kingma and L. J. Ba, "Adam: A Method for Stochastic Optimization," 2015. Accessed: Apr. 19, 2026.

[14] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, PMLR, Jun. 2015, pp. 448–456. Accessed: Apr. 18, 2026.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014, ISSN: 1533-7928. Accessed: Apr. 18, 2026.

[16] T. Hastie, R. Tibshirani, and J. Friedman, "Model Assessment and Selection," in *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, T. Hastie, R. Tibshirani, and J. Friedman, Eds., New York, NY: Springer, 2009, pp. 219–259, ISBN: 978-0-387-84858-7. DOI: 10.1007/978-0-387-84858-7_7 Accessed: Apr. 10, 2026.

[17] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, ISSN: 1533-7928. Accessed: Apr. 19, 2026.

[18] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, vol. 24, Curran Associates, Inc., 2011. Accessed: Apr. 19, 2026.

[19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19, New York, NY, USA: Association for Computing Machinery, Jul. 2019, pp. 2623–2631, ISBN: 978-1-4503-6201-6. DOI: 10.1145/3292500.3330701 Accessed: Apr. 1, 2026.

[20] C. Rodriguez, L. Degioanni, L. Kameni, R. Vidal, and G. Neglia, *Evaluating the Energy Consumption of Machine Learning: Systematic Literature Review and Experiments*, Aug. 2024. DOI: 10.48550/arXiv.2408.15128 arXiv: 2408.15128 [cs]. Accessed: Mar. 11, 2026.

[21] M. F. Stefan Aeberhard, *Wine*, 1992. DOI: 10.24432/C5PC7J Accessed: Mar. 19, 2026.

[22] I-Cheng Yeh, *Default of Credit Card Clients*, 2009. DOI: 10.24432/C55S3H Accessed: Mar. 19, 2026.

[23] P. Baldi, P. Sadowski, and D. Whiteson, "Searching for Exotic Particles in High-Energy Physics with Deep Learning," *Nature Communications*, vol. 5, no. 1, p. 4308, Jul. 2014, ISSN: 2041-1723. DOI: 10.1038/ncomms5308 arXiv: 1402.4735 [hep-ph]. Accessed: Mar. 19, 2026.

[24] D. Whiteson, *HIGGS*, 2014. DOI: 10.24432/C5V312 Accessed: Mar. 19, 2026.

[25] D. R. Cox, "The Regression Analysis of Binary Sequences," *Journal of the Royal Statistical Society. Series B (Methodological)*, vol. 20, no. 2, pp. 215–242, 1958, ISSN: 0035-9246. JSTOR: 2983890. Accessed: Apr. 8, 2026.

[26] R. Desislavov, F. Martínez-Plumed, and J. Hernández-Orallo, "Trends in AI inference energy consumption: Beyond the performance-vs-parameter laws of deep learning," *Sustainable Computing: Informatics and Systems*, vol. 38, p. 100 857, Apr. 2023, ISSN: 22105379. DOI: 10.1016/j.suscom.2023.100857 Accessed: Apr. 10, 2026.

[27] A. Bagnall and G. C. Cawley, *On the Use of Default Parameter Settings in the Empirical Evaluation of Classification Algorithms*, Mar. 2017. DOI: 10.48550/arXiv.1703.06777 arXiv: 1703.06777 [cs]. Accessed: Apr. 10, 2026.

[28] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, no. 85, pp. 2825–2830, 2011, ISSN: 1533-7928. Accessed: Mar. 19, 2026.

[29] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, Dec. 2019. DOI: 10.48550/arXiv.1912.01703 arXiv: 1912.01703 [cs]. Accessed: Mar. 19, 2026.

[30] B. Ghosh, *Skorch: The Bridge Between PyTorch and Scikit-Learn*, Sep. 2024. Accessed: Mar. 26, 2026.

[31] B. Courty et al., *Mlco2/codecarbon: V2.4.1*, Zenodo, May 2024. DOI: 10.5281/zenodo.11171501 Accessed: Apr. 15, 2026.

[32] Anthropic, *Claude code*, AI-assisted software development tool, 2025. Accessed: Apr. 20, 2026. [Online]. Available: https://claude.ai/code

List of Figures

List of Tables